

Aegis: Distributed Telemetry Engine 🏰

Solving the "Context Gap" in Distributed Observability

Aegis is a high-fidelity telemetry engine that moves beyond passive monitoring. It captures the "Why" behind system anomalies by maintaining a rolling 60-second window of deep system state at the edge, flushing it to a Scala-powered brain for real-time correlation and LLM-driven diagnostic briefings.

🏰 The Core Philosophy

The Edge (Go) – *The Intelligent Sentinel*

Instead of a passive collector, the Aegis Go agent acts as a sentinel. It uses a **Ring-Buffer Strategy** to store the last 60 seconds of high-fidelity state (syscalls, network packets, stack traces) locally. Data is only flushed when a threshold is hit or when explicitly requested by the Brain.

The Brain (Scala) – *The Global State Map*

Utilizing the **Actor Model**, the Scala cluster treats incoming telemetry as a continuous stream of events. It runs sliding-window analysis to correlate events across different agents (e.g., matching a latency spike on Agent A with a connection drop on Agent B).

The Insight (RAG Layer) – *Diagnostic Briefings*

When an anomaly is detected, Aegis retrieves the 60-second buffer, indexes it into a vector store, and provides a **Diagnostic Briefing** via an LLM, bridging the gap between metrics and root causes.

🏰 Project Structure

- **agent/** : Go-based intelligent sentinels.
 - **cluster/** : Scala/Akka-based correlation brain.
 - **proto/** : High-performance gRPC/Protobuf definitions.
 - **docs/** : Engineering proposals and architectural blueprints.
-

🏰 Interactive Simulator

Visualize how Aegis handles massive scale and anomaly detection under load.

```
# Run the Aegis Simulator to test scaling and chaos scenarios
go run ./simulator/main.go --agents 1000 --chaos network-partition
```

Simulation Scenarios:

- **Scaling:** Monitor how the Scala cluster handles backpressure from 10,000+ agents.
- **Zero-Drop:** Test local caching on Go agents during network partitions.

- **Anomaly Flush:** Trigger an alert and watch the high-fidelity buffer transmission.
-

▣ Documentation

For the full engineering proposal, see <docs/architecture.md>.

- **System Philosophy:** Reactive vs. Passive tracing.
 - **C4 Diagrams:** Multi-level component visualization.
 - **Communication Contract:** gRPC/Protobuf rationale.
 - **Scaling & Failure:** Backpressure and Zero-Drop strategies.
-

▣ License

MIT © 2026 Aegis Team

▣ Project Roadmap

- ☒ **Phase 1:** Distributed Cluster & Agent Topology Scaffold.
- ☒ **Phase 2:** Core Telemetry Infrastructure & Protobufs.
- ☐ **Phase 3:** Centralized API & Microservices Integration.
- ☐ **Phase 4:** Frontend Dashboard Implementation.